

Question No:7

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A .

Note: Subarray means contiguous elements with at-least one element in it.

Constraints

$$1 \leq N \leq 2000$$

$$0 \leq |A_i| \leq 10^9$$

Input Format

The first line of the input contains a single integer N , the total number of elements in array A .

The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

4
5 -2 7 -3

EXPECTED OUTPUT

17

Test Case 2

INPUT (STDIN)

15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9

EXPECTED OUTPUT

2021

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX 2000
5
6  long long sums[MAX * (MAX + 1) / 2];
7
8  int compare(const void *a, const void *b) {
9      long long x = *(long long *)a;
10     long long y = *(long long *)b;
11
12     if (x < y)
13         return -1;
14     if (x > y)
15         return 1;
16     return 0;
17 }
18
19 int main() {
20     int N;
21     scanf("%d", &N);
22
23     long long A[MAX];
24
25     for (int i = 0; i < N; i++) {
26         scanf("%lld", &A[i]);
27     }
28
29     int count = 0;
```

```

30
31 // Generate every possible subarray
32 for (int i = 0; i < N; i++) {
33     long long current = 0;
34     long long maxSum = A[i];
35
36     for (int j = i; j < N; j++) {
37
38         // Kadane's algorithm
39         if (j == i) {
40             current = A[j];
41         } else {
42             if (current + A[j] > A[j])
43                 current = current + A[j];
44             else
45                 current = A[j];
46         }
47
48         if (current > maxSum)
49             maxSum = current;
50
51         sums[count] = maxSum;
52         count++;
53     }
54 }
55
56 // Sort all maximum subarray sums
57 qsort(sums, count, sizeof(long long), compare);
58
59 // Add only unique values
60 long long total = 0;
61
62 for (int i = 0; i < count; i++) {
63     if (i == 0 || sums[i] != sums[i - 1]) {
64         total += sums[i];
65     }
66 }
67
68 printf("%lld\n", total);
69
70 return 0;
71 }

```

Output:

Test Case-1:

```
4
5 -2 7 -3
17
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA_Practice\Hashing
```

Test Case-2:

```
15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
2021
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA_Practice\Hashing
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236